

ACCIDENT DETECTION AND NOTIFICATION SYSTEM USING AWS



Mini project submitted in partial fulfillment of the requirement for the award of the degree
of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Under the esteemed guidance of

Pratima A. Raut
Assistant Professor

By

MEDABOINA BHARGAVI	(23U15A0506)
ERUMALLA AKHIL KUMAR	(22U11A0528)
JANGILI SANDHYA	(22U11A0535)
MEKALA STALIN	(22U11A0551)



Department of Computer Science and Engineering

Samskruti College of Engineering and Technology (Autonomous)

Approved by AICTE & Affiliated to JNTUH Accredited by NBA and NAAC with A Grade

Accredited by the NBA Kondapur(V), Ghatkesar (M), Medchal District, Telangana – 501301

MAY – 2025

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the B.Tech Mini Project report entitled **“ACCIDENT DETECTION AND NOTIFICATION SYSTEM USING AWS”** is a bonafide work done by **MEDABOINA BHARGAVI (23U15A0506), ERUMALLA AKHIL KUMAR (22U11A0528), JANGILI SANDHYA (22U11A0535), MEKALA TALIN (22U11A0551)** in partial fulfillment of the requirement of the award for the degree of Bachelor of Technology in **“Computer Science and Engineering”** from Jawaharlal Nehru Technological University, Hyderabad during the year 2024-2025.

Project Guide
Pratima A. Raut
Assistant professor

HoD-CSE
Dr V ShivaNarayan Reddy
Assoc Professor

External Examiner

Samskruti College of Engineering and Technology (Autonomous)

Approved by AICTE & Affiliated to JNTUH, Accredited by NBA and NAAC with A Grade

Kondapur(V), Ghatkesar (M), Medchal District, Telangana – 501301

Department of Computer Science and Engineering



DECLARATION BY THE CANDIDATE

We, **MEDABOINA BHARGAVI (23U15A0506), ERUMALLA AKHIL KUMAR (22U11A0528), JANGILI SANDHYA (22U11A0535), MEKALA STALIN, , (22U15A0551)** respectively, hereby declare that the

project report entitled “**ACCIDENT DETECTION AND NOTIFICATION SYSTEM USING AWS**” is done under the guidance of **Mrs. PRATIMA A. RAUT, Associate Prof.**, Department of Computer Science and Engineering, Samskruti College of Engineering and Technology, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering.**

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

MEDABOINA BHARGAVI (23U15A0506)

ERUMALLA AKHIL KUMAR (22U11A0528)

JANGILI SANDHYA (22U11A0535)

MEKALA STALIN (22U11A0551)

Department of CSE,

Samskruti College of Engineering and Technology, Ghatkesar.

ACKNOWLEDGEMENT

It is with profound gratitude and sincere appreciation that I extend my heartfelt thanks to all those who have played a significant role in the successful completion of this undergraduate project.

First and foremost, I express my deep sense of respect and gratitude to our **Honourable Chairman, Mr. A V Ramana Reddy**, for his constant encouragement and for nurturing a culture of academic excellence and innovation within the institution.

I would also like to express my sincere thanks to **Dr. M. Ramakanth Reddy, Director**, for his visionary leadership and continued support, which have provided the ideal environment and motivation for academic pursuits such as this project.

My heartfelt appreciation goes to **Dr. P Janaki Ramulu, Principal**, for his steadfast guidance, infrastructural support, and encouragement that have helped bring this project to successful fruition.

I am deeply grateful to **Dr. V Shiva Narayana Reddy, Head of the Department of Computer Science and Engineering**, for his academic leadership, valuable feedback, and continuous support throughout the duration of this project.

A special note of gratitude is reserved for my project guide, **Ms. Pratima A Raut, Associate Prof.**, whose expert supervision, insightful suggestions, and dedicated mentorship have been instrumental in shaping the direction and outcome of this work.

Lastly, I am ever grateful to my **parents and family** for their unconditional love, encouragement, and moral support. Their faith in me has been my greatest strength throughout this journey.

With genuine appreciation, I acknowledge every individual who has, in one way or another, contributed to the successful completion of this project.

With warm regards,

MEDABOINA BHARGAVI (23U15A0506)

ERUMALLA AKHIL KUMAR (22U11A0528)

JANGILI SANDHYA (22U11A0535)

MEKALA STALIN (22U11A0551)

**Department of Computer Science and Engineering Samskruti College
of Engineering and Technology**

ABSTRACT

With the constant advancements in the automotive industry and the increase in traffic volume, chances of fatality in road accidents have increased. This poses a dire need to develop a good and reliable solution to reduce the fatalities. One way to achieve this goal is to build a system that detects road accidents and notifies medical services as soon as possible. The aim of this project is to provide a robust accident detection and notification system where the data collected from the accident site is communicated to an emergency medical service provider as well as the victim's emergency contacts such as family members or close friends. The system detects a crash and sends a notification, a text message and an email using AWS to the concerned parties with the latency of approximately 20 seconds and this was achieved by using an internet connection with a speed of 3.76 MBPS. Since AWS is reliable, scalable and fast, the cloud infrastructure provides a better platform for the system.

LIST OF FIGURES, DIAGRAMS, GRAPHS, SCREENSHOTS

Figure No	Description	Page No
Fig - 1.1	Fatality rate Pie Chart	1
Fig – 4.1	Data Flow Diagram	18
Fig – 4.2.1	Use case Diagram	20
Fig – 4.2.2	Class Diagram	21
Fig – 4.2.3	Sequence Diagram	21
Fig – 4.2.4	Activity Diagram	22
Fig – 4.2.5	Deployment Diagram	24
Fig – 5.1.2	History of Python	29
Fig – 5.2	Installation of Python	42

LIST OF SCREENSHOTS

Figure No	Description	Page No
Fig – 8.1	Home Page	64
Fig – 8.2	User Login	64
Fig – 8.3	Registration Form	65
Fig – 8.4	Forgot Password	65
Fig – 8.5	Contact Information	66
Fig – 8.6	Analysis	66
Fig – 8.7	Edit Profile	67
Fig – 8.8	Emergency Form	67
Fig – 8.9	Scan Emergency QR Code	68
Fig – 8.10	Accident Type Form	68

Table of Contents

S.No	Content	Page No.
1.	Title Page	i
2.	Certificate	ii
3.	Declaration	iii
4.	Acknowledgement	iv
5.	Abstract	v
6.	List of figures/diagrams/graphs/Screen shots	vi

Chapter 1: Introduction

1.1	Overview of the Project	2
1.2	Problem Statement	2
1.3	Objectives of the Project	2
1.4	Scope of the Project	2
1.5	Methodology / SDLC Model Adopted	3
1.6	Organization of the Report	3-4

Chapter 2: Literature Survey

2.1	Review of Existing System	6
2.2	Limitations of Existing Approaches	6
2.3	Need for the Proposed System	6-7
2.4	Comparative Study	7
2.5	Summary	7

Chapter 3: System Analysis

3.1	Feasibility Study	9
	• Technical Feasibility	
	• Economic Feasibility	
	• Operational Feasibility	
	• Time & Cost Estimation	
3.2	Software Requirements Specification (SRS)	9
3.3	Functional and Non-Functional Requirements	9-10

Chapter 4: System Design

4.1	System Architecture	12
4.2	Database Design (ER Diagram) / Data Flow Diagrams / Workflow	13
4.3	UML Diagrams (Use Case, Class, Sequence, Activity)	14 - 18
4.4	User Interface Design	19
4.5	Design Standards Followed (IEEE, ISO, etc.)	19
4.6	Safety & Risk Mitigation Measures	19

Chapter 5: Implementation

5.1	Technology Stack	21
5.2	Module-wise Implementation	21
5.3	Code Integration Strategy	21
5.4	Sample Code Snippets	22 - 36
5.5	Coding Standards Followed (if any)	37

Chapter 6: Testing

6.1	Testing Strategy	39
6.2	Unit Testing	39
6.3	Integration Testing	40
6.4	System Testing	40-41
6.5	Test Cases and Results	42
6.6	Bug Reporting and Tracking	42
6.7	Quality Assurance Standards (If any)	42

Chapter 7: Results and Discussion

7.1	Output Screenshots	48-53
7.2	Results Interpretation	54
7.3	Performance Evaluation	54
7.4	Comparative Results	54

Chapter 8: Conclusion and Future Scope

8.1	Summary of work done	56
8.3	Limitations	56
8.4	Challenges Faced	56
8.5	Future Enhancements	56

Chapter 9: References

•	Technical Publications (Journals and conference papers)	58-60
•	Websites and forums details	61

Chapter 10: Appendices

A. SDLC Forms (SRS, Feasibility, Test Reports, etc.)	63
B. Cost Estimation Sheets (if available)	64
C. Ethical Considerations & Consent	65
D. Plagiarism Report (Turnitin /URKUND / DrillBit Certificate)	65
E. Source Code Repository / Deployment Links (eg. GITHUB / Drive links)	65
F. Additional Screenshots or User Manuals	65-66
H. Journal / Conference paper published on project	66
I. Proof of Certificate - participation / Prize (if attended any project expo)	66

CHAPTER: 1

INTRODUCTION

1.1 Overview of the Project

This project aims to reduce road accident fatalities by developing a real-time accident detection and notification system. It alerts emergency services and the victim's contacts via SMS and email, using AWS for reliable, scalable, and low-latency communication. Testing showed an average notification time of 20 seconds, demonstrating its potential to improve emergency response and save lives.

1.2 Problem Statement

With the increasing number of vehicles on the road and the rising frequency of traffic accidents, there is a growing concern about the delay in providing timely medical assistance to accident victims. Current systems often lack real-time accident detection and efficient communication with emergency responders and family members, leading to higher fatality rates. There is a critical need for a reliable, fast, and automated accident detection and notification system that can promptly alert emergency services and the victim's contacts to improve the chances of survival and minimize the consequences of delayed response.

1.3 Objectives of the Project

The primary goal of this project is to create a smart accident detection and notification system that quickly alerts emergency services and the victim's contacts. It uses an accelerometer to detect accidents in real time and sends a notification, SMS, and email via AWS. With an average alert time of 20 seconds on a 3.76 Mbps connection, AWS ensures fast, reliable, and scalable communication for timely emergency response.

1.4 Scope of the Project

This project aims to design a system that detects road accidents using an accelerometer and instantly notifies emergency contacts and services via notification, SMS, and email. Powered by AWS cloud services, it ensures fast and reliable communication. The focus is on using affordable components and stable internet to reduce emergency response time and save lives. Key functions include accident detection, location-based alerts, and efficient data handling through AWS.

1.5 Methodology/ SDLC Model Adopted

This project followed the Waterfall Model of SDLC, ideal for structured embedded and cloudbased systems:

1. **Requirement Analysis:** Identified the need for real-time accident detection and quick notifications using an accelerometer, microcontroller, internet, and AWS.
2. **System Design:** Designed an architecture to collect sensor data and send alerts via AWS IoT Core, SNS, and Lambda.
3. **Implementation:** Integrated hardware and software; the accelerometer was programmed to detect crashes, and AWS was set up for alert delivery.
4. **Testing:** Conducted simulated crash tests; average response time was around 20 seconds on a 3.76 Mbps connection.
5. **Deployment:** System deployed with stable internet and hosted on AWS for reliable performance.
6. **Maintenance:** Post-deployment improvements were made to enhance latency and reliability.

1.6 Organization of the Report

1.6.1. Overview

This report presents a smart accident detection and notification system using IoT and AWS cloud services. It aims to reduce road accident fatalities by ensuring rapid alerts to emergency services and the victim's contacts through automated processes.

1.6.2. Key Objectives

- Enable real-time detection of road accidents using sensor data.
- Notify emergency services and contacts via SMS, email, and cloud alerts.
- Utilize AWS for reliable, scalable, and low-latency communication.
- Reduce emergency response time to improve survival chances.

1.6.3. System Architecture

Sensors & Detection: Accelerometer and microcontroller monitor sudden impact.

- **Cloud Integration:** AWS IoT Core, Lambda, SNS, and SES for processing and alerting.
- **Alert Mechanism:** Sends real-time SMS, email, and notifications upon crash detection.
- **Connectivity:** Requires stable internet (~3.76 Mbps) for timely data transmission.

1.6.5. Privacy & Ethics

Collects only essential crash-related data.

- Ensures encrypted, secure communication via AWS.
- Operates with user consent for emergency contact alerts.
- Designed solely for emergency use, avoiding constant tracking.
- Follows standard data protection and ethical guidelines.

1.6.6. Conclusion

The system offers a reliable and scalable solution for accident detection and emergency response. With AWS at its core, it ensures fast communication and holds potential for future enhancements in coverage, precision, and integration.

CHAPTER: 2

LITERATURE

SURVEY

2.1 Review of Existing System

In recent years, various research papers have explored accident detection and notification systems using different technologies. One study utilized data from NHSTA (USA) to test algorithm accuracy for predicting accident severity. Another approach by Ahirrao et al. involved a dedicated collision detection circuit using LDR and photodiode sensors, combined with an Android app for notifications. Other works have demonstrated the use of Android smartphone sensors for crash detection. Systems like e-Notify require an onboard unit (OBU) to detect and alert emergency services, but cost and scalability limit widespread adoption. Additionally, the European Union's eCall system, mandated for all vehicles manufactured after 2015, automatically contacts emergency services (such as 112 or 999 in the UK) upon detecting a crash. While each method offers valuable insights, challenges remain in affordability, infrastructure, and rapid response effectiveness.

2.2 Limitation of Existing Approach

- costly and it's difficult to equip every vehicle
- No Information available of riders or drivers
- Not able to notify beloved ones
- Requires Police action for Information which will be time consuming

2.3 Need for Proposed System

The system presented in this work successfully sends notifications, SMS, and emails to concerned parties using Amazon Web Services (AWS), achieving an average latency of approximately 20 seconds with a 3.76 Mbps internet connection. Leveraging AWS ensures a reliable, scalable, and fast cloud infrastructure, making it ideal for time-critical applications. The system proves especially valuable in remote or isolated areas such as mountainous regions, villages, or interior roads—particularly during nighttime—where human assistance may not be immediately available. In such scenarios, this automated solution can be life-saving. To ensure broad applicability and effectiveness in reducing fatality rates, the system is designed to be robust, adaptable, and installable in any vehicle. A simple web application has also been developed to receive push notifications, such as “An accident is detected”, triggered via AWS

Simple Notification Service (SNS). Future enhancements may include displaying additional information such as accident details and vehicle owner data. Furthermore, a website built with HTML, CSS, and JavaScript serves as a frontend interface for hospitals and medical centers. It displays the most recent accident details along with a comprehensive history of all recorded incidents. The entire system is implemented using Python with the Django framework and MySQL as the backend database, ensuring a stable and efficient platform for real-time accident management.

2.4 Comparative Study

Approach	Pros	Cons
GSM-Based Systems	Simple setup, uses existing mobile networks	High latency (~60 sec), no real-time processing, low scalability, limited to SMS alerts
Firebase-Based Systems	Real-time syncing, supports email & app notifications, easy to implement for small apps	Lacks native SMS support, limited hardware integration, less scalable for large systems
Local Server-Based Systems	Offers low latency when optimized, independent of cloud	Requires setup & maintenance, poor scalability, vulnerable to power/internet outages
Proposed AWS-Based System	Low latency (~20 sec), scalable, reliable, secure, supports multi-channel alerts, cost-effective	Requires AWS expertise, internet dependent, potential costs at scale (but still efficient overall)

2.5 Summary

With the rise in road traffic, the number of serious accidents and fatalities has also increased. This project proposes a smart accident detection and alert system that aims to reduce such fatalities by enabling rapid emergency response. It uses accelerometer data to detect collisions in real time and sends alerts—including SMS and email notifications—to emergency services and the victim's contacts. Leveraging Amazon Web Services (AWS), the system ensures alerts are delivered within approximately 20 seconds, offering a fast, reliable, and scalable solution.

CHAPTER: 3

SYSTEM ANALYSIS

3.1 Feasibility Study

- **Technical Feasibility** – Requires CCTV footage, AI models (CNNs), and secure databases.
- **Economic Feasibility** – High initial costs, but reduces long-term law enforcement expenses.
- **Operational Feasibility** – Enhances crime prevention, suspect tracking, and investigation efficiency.
- **Legal & Ethical Feasibility** – Addresses privacy concerns, data security, and AI biases.

3.2 Software Requirements Specifications (SRS)

- **System Interfaces:** Accelerometer input, AWS IoT Core, SNS, SES, Lambda, web dashboard.
- **User Interfaces:** Web portal for hospitals, push notification app, admin panel.
- **Hardware Interfaces** Microcontroller (ESP32/Arduino), sensors, internet/GSM module.
- **Software Interfaces:** Python, Django, AWS (IoT Core, Lambda, SNS, SES), MySQL.

3.3 Functional and Non-functional Requirements

3.3.1 Functional requirements

- Detects crashes using accelerometer data.
- Sends alerts via SMS, email, and app using AWS services.
- Logs and displays accident data on a web interface.
- Admin dashboard for managing users and accident history.

3.3.2 Non-Functional Requirements

➤ HARDWARE REQUIREMENTS

MINIMUM (Required for Execution)		MY SYSTEM (Development)
System	Pentium IV 2.2 GHz	i3 Processor 5 th Gen
Hard Disk	20 Gb	512 Gb
Ram	1 Gb	4 Gb

➤ SOFTWARE REQUIREMENTS

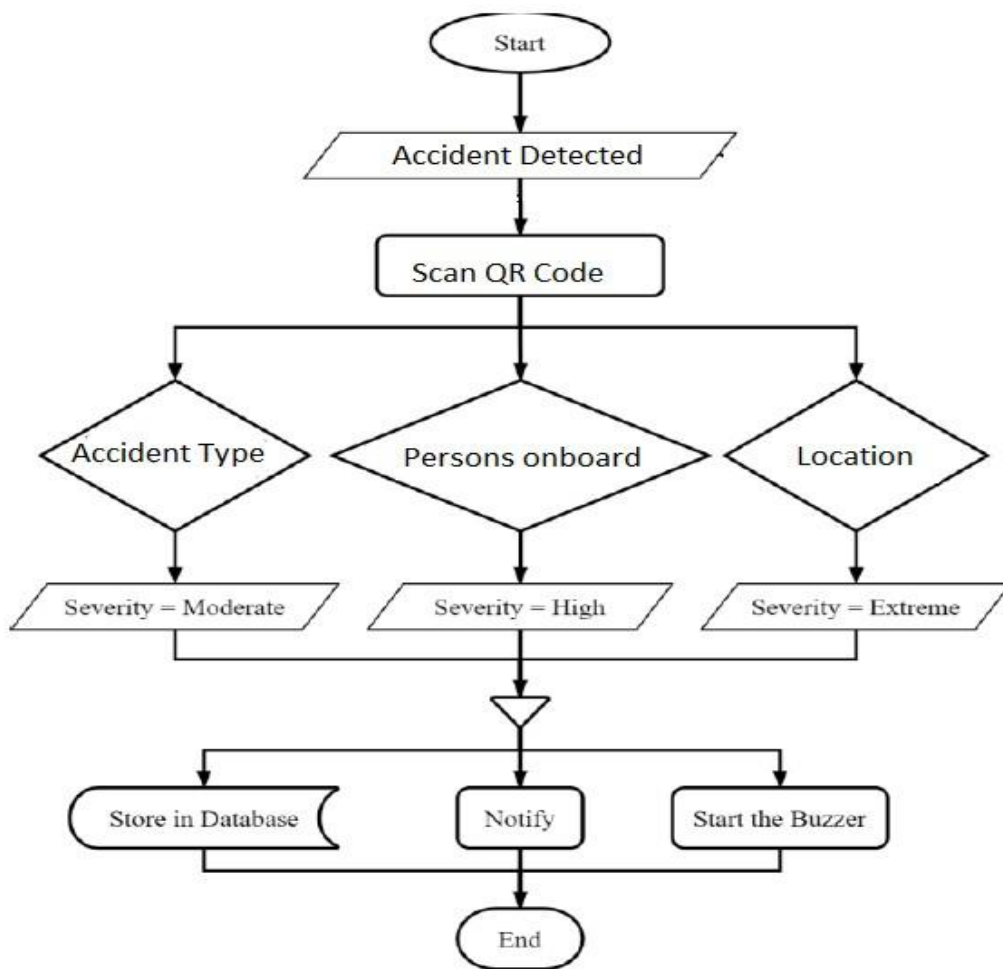
Operating System	Windows 10/11
Development Software	Python 3.10
Programming Language	Python
Domain	Image Processing & Cloud Computing
Integrated Development Environment (IDE)	Visual Studio Code
Front End Technologies	HTML5, CSS3, Java Script
Back End Technologies or Framework	Django
Database Language	SQL
Database (RDBMS)	MySQL
Database Software	WAMP or XAMPP Server
Web Server or Deployment Server	Django Application Development Server
Design/Modelling	Rational Rose

CHAPTER: 4

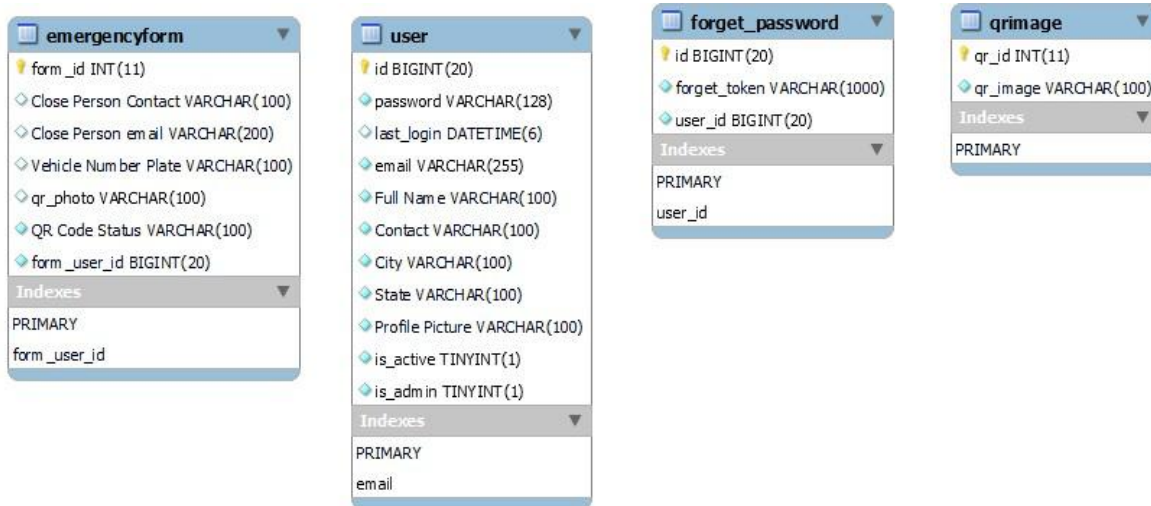
SYSTEM

DESIGN

4.1 System Architecture



4.2 Database Design / Data Flow Diagrams



A **Data Flow Diagram (DFD)**, also known as a **bubble chart**, is a graphical tool used to model a system's processes, data flow, and external interactions. It illustrates how input data is transformed into output through various processes. DFDs can be used at multiple abstraction levels and help visualize system components, data usage, and flow transformations effectively.

UML Diagrams (Use Case, Class, Sequence, Activity)

- **Definition** – UML (Unified Modeling Language) is a standardized visual language used in object-oriented software engineering to model software systems.
- **Components** – Consists of a Meta-model and Notation; managed by the Object Management Group (OMG).
- **Purpose** – Used for specifying, visualizing, constructing, and documenting software system artifacts and business models.
- **Significance** – Captures best engineering practices for modeling large, complex, and object-oriented systems using graphical notations.

Goals of UML

1. Provide a ready-to-use, expressive visual modeling language.
2. Enable extensibility and specialization of core concepts.
3. Remain independent of specific programming languages and processes.
4. Offer a formal basis for understanding models.
5. Promote the development of object-oriented tools.
6. Support advanced concepts like collaborations, frameworks, and components.
7. Integrate industry-proven best practices.

4.3 UML Diagrams

□ Use Case Diagram:

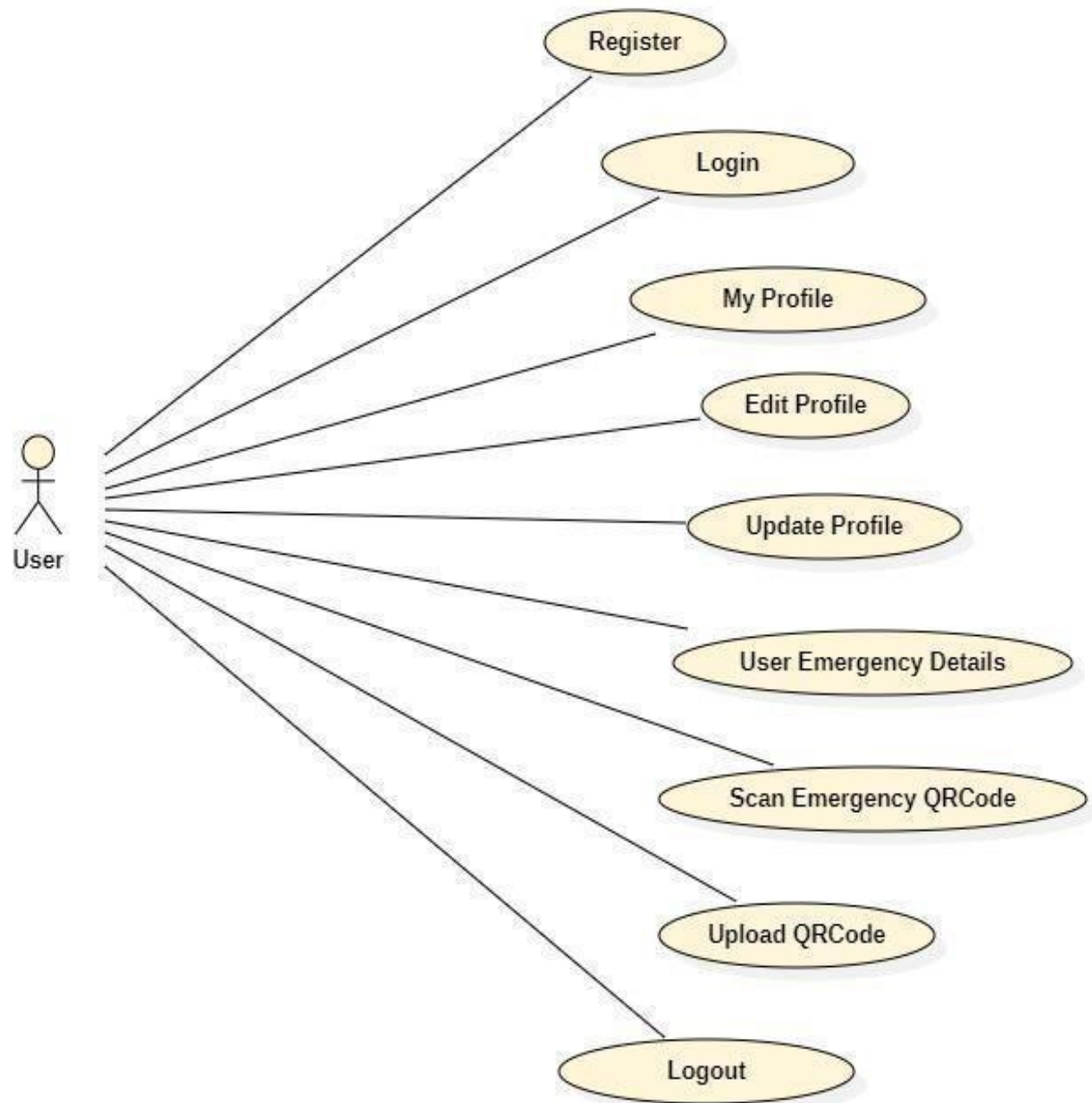


FIG 4.3.1: Use case UML diagram for USER

□ Class Diagram

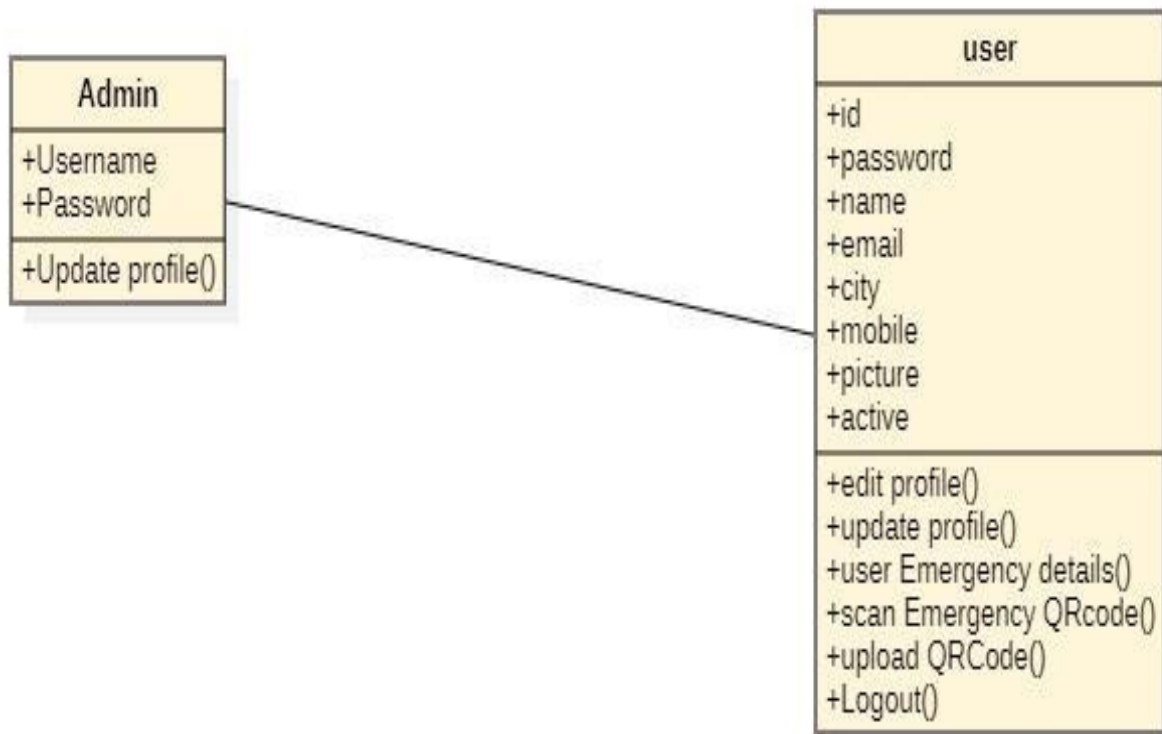


FIG 4.3.3: Class UML diagram

□ Sequence Diagram

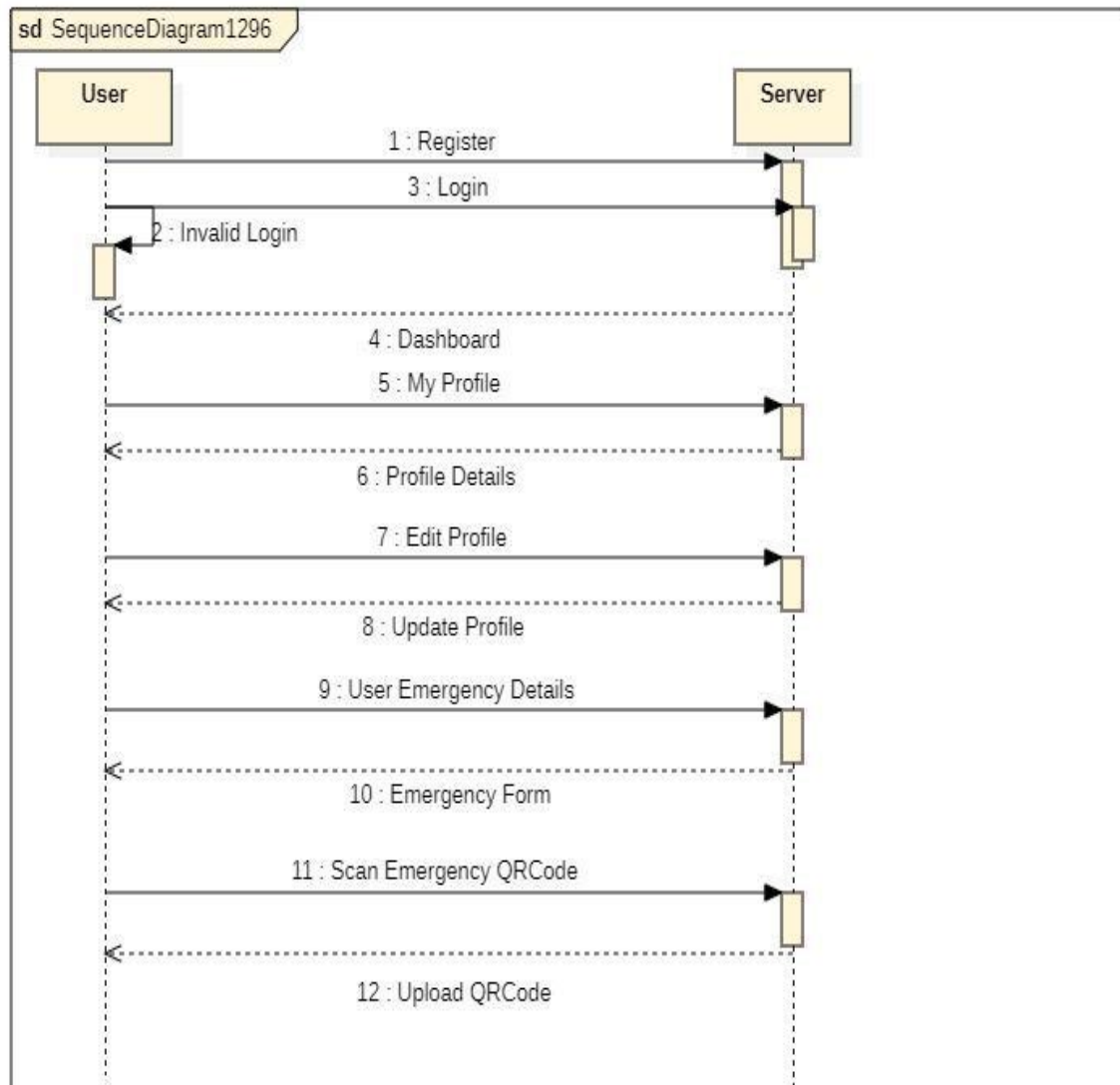


FIG 4.3.4: Sequence UML diagram

Activity Diagram

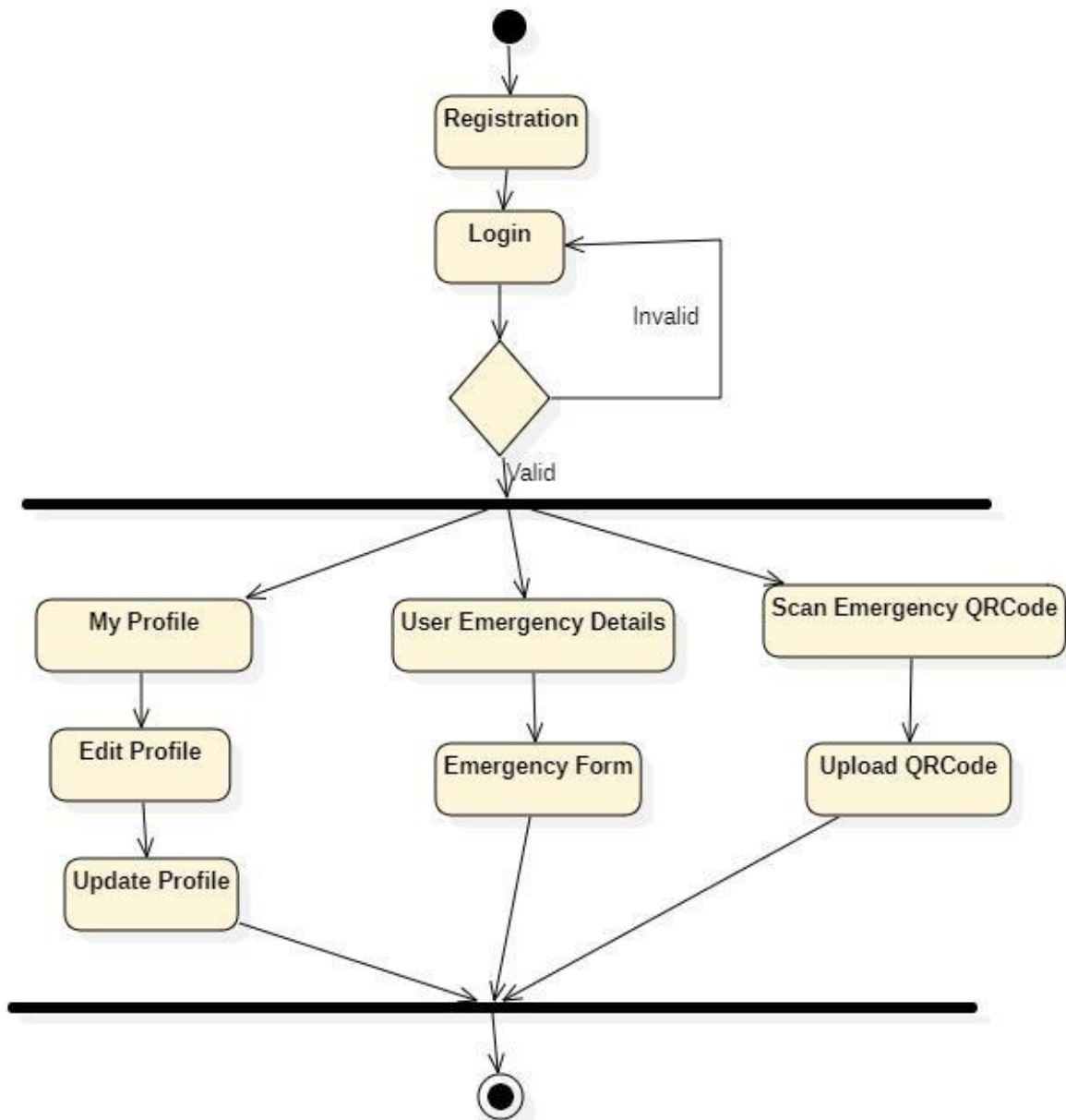


FIG 4.3.5: Activity UML diagram for ADMIN

Deployment Diagram

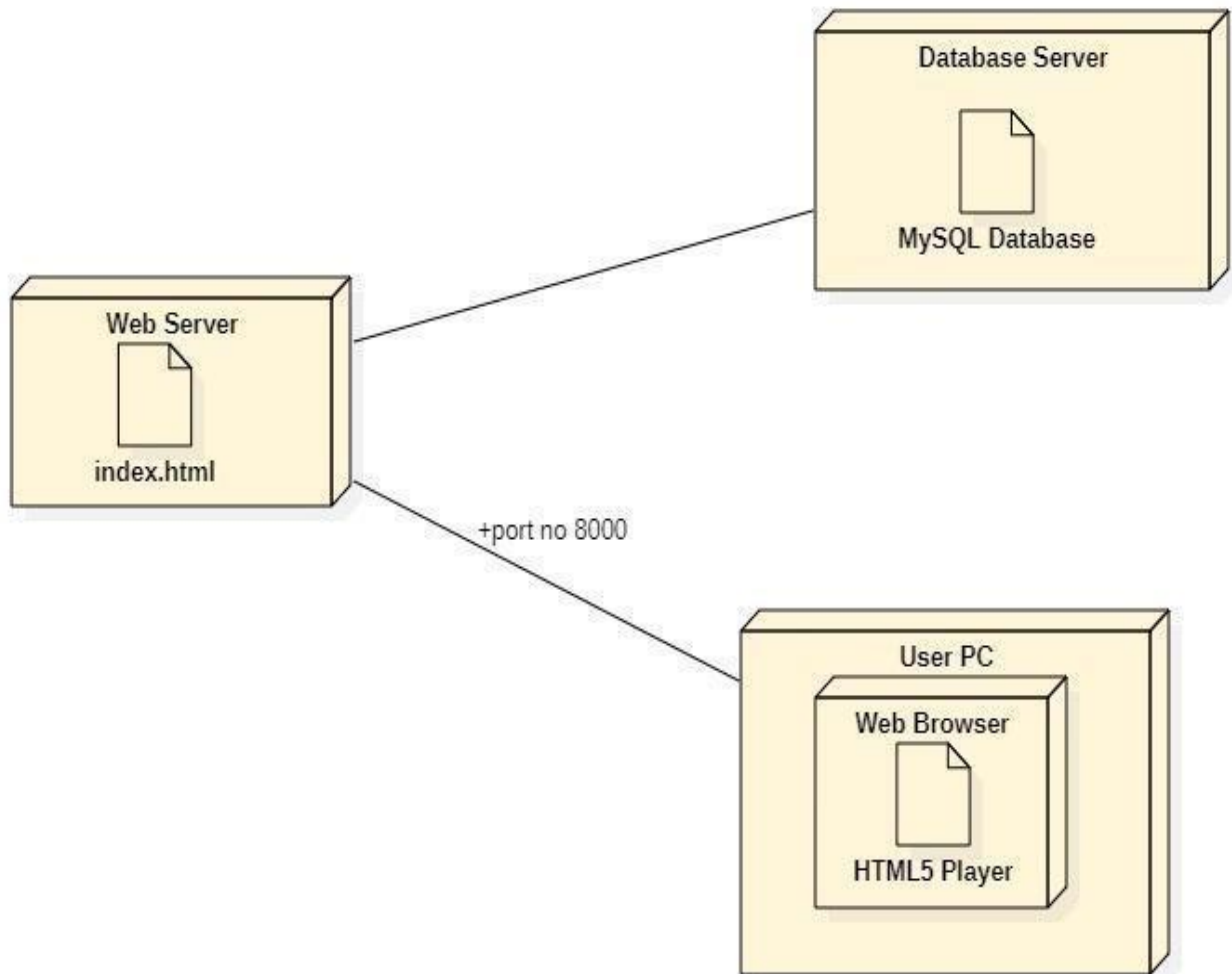


FIG 4.3.7: Deployment UML diagram

4.4 User Interface Design

Login and registration using AWS Cognito

- Real-time status dashboard with GPS location and alerts
- Manage emergency contacts with SMS/Email preferences
- View accident history with map and export options
- Emergency operator panel with live incident feed and dispatch control

4.5 Design Standards Followed

- Followed IEEE 1016 (Software Design Description Standard)
- Testing based on IEEE 829 documentation guidelines
- Ensured quality via ISO/IEC 25010 (reliability, maintainability)
- Data security aligned with ISO/IEC 27001 cloud standards

4.6 Safety & Risk Mitigation

- Automatic crash detection and instant alerting
- Real-time emergency notifications to responders and contacts
- Secure AWS-based cloud storage for all incident data
- Redundant SMS and Email alert mechanisms to ensure delivery

CHAPTER:5

IMPLEMENTATION

5.1 Technology Stack

- **Hardware** → Raspberry Pi 3B, MPU-6050 accelerometer, pressure sensor, buzzer
- **Software** Python 3, AWS CLI, boto3 for AWS integration
- **AI Models** → Custom acceleration and DeltaV-based accident severity algorithm
- **Database** → AWS DynamoDB for accident and vehicle record storage
- **Security** → AWS IAM roles, secure key storage using CLI credential configuration
- **Frontend** → Android app and web interface (HTML, CSS, JS) for alerts and monitoring
- **Backend** → Python-based detection system integrated with AWS services (SNS, PinPoint, DynamoDB)

5.2 Module-wise Implementation

- **Sensor Calibration** → Calibrates MPU-6050 for accurate acceleration readings using offset correction
- **Accident Detection** → Identifies crashes via acceleration $>5g$ and computes DeltaV for severity
- **Severity Estimation** → Uses Butterworth filter and integration to classify accident impact (High/Extreme)
- **Alert System** → Triggers AWS SNS (push/email) and PinPoint (voice) based on detection
- **Database Management** → Dynamically logs data to DynamoDB (Accidents and Vehicle tables) • **Emergency Dashboard** → Displays incident info on app/website for real-time response identification.

5.3 Code Integration Strategy

Each module (sensor reading, detection, AWS alerts, data logging) tested independently → Integrated on Raspberry Pi using a modular Python design → AWS services accessed via boto3 APIs for seamless communication → Android and web platforms connected to backend for real-time monitoring and historical access.

5.4 Sample Code Snippets

```
from django.shortcuts import render,redirect import

boto3 from boto3.session import Session from

django.contrib.auth import authenticate,login,logout

from django.contrib.auth.decorators import

login_required from django.contrib.auth.models

import User from userapp.models import *

from django.db import IntegrityError

from django.contrib import messages

import qrcode from PIL import Image,

ImageDraw from io import BytesIO

from django.core.files import File

from django.core.mail import

send_mail import requests from

pyzbar.pyzbar import decode import

boto3 from botocore.exceptions import

ClientError import uuid from

django.conf import settings

from .models import Profile from

django.core.mail import send_mail

import cv2 import json
```



```

# Create your views here. def

log_user(request):    if

request.method == 'POST':

    print('check1')    email =

request.POST.get('email')

    print('check2')    password =

request.POST.get('password')

    print('check3')    user =

authenticate(request=None,email

=email, password=password)

    print('check4')    if user is

not None:

    login(request,user)

    print('loginnnnnn')    return

redirect('user_dashboard')

    else:

        print(email,password)    return

redirect('log_user')    return

render(request,'main/home-user-login.html')

def register(request):

    try:

```

```

        if request.method == 'POST':

            fullname=request.POST.get('fullname')

            email=request.POST.get('email')

            contact=request.POST.get('contact')

            city=request.POST.get('city')

            state=request.POST.get('state')

            pic=request.FILES['pic']

            password=request.POST.get('password')

            user = MyUser.objects.create_user( email,

            password)          user.fullname = fullname

            user.contact = contact          user.state =

            state          user.city = city

            user.image = pic

            user.save()

            ftoken = str(uuid.uuid4())          profile =

            Profile.objects.create(user=user,forget_token=ftoken)

            messages.success(request,'Registration Successfull')

            except IntegrityError:

            messages.warning(request,'Email Has Already Been Taken')

            return redirect('register')          return

            render(request,'main/home-user-register.html')

```

```

def forget_password(request):

    # try:

    if request.method == 'POST':
        email = request.POST.get('email')

    user = MyUser.objects.get(email=email)

    profile = Profile.objects.get(user=user)

    user_email = user.email

    print(user_email)    ftoken =

    profile.forget_token

    mail_message = f'Hey Your Reset Password Link is
http://127.0.0.1:8000/changepass/{ftoken}/'    print(mail_message)

    send_mail('Password Request',mail_message,settings.EMAIL_HOST_USER,[user_email],fail_silently=
Reset
False)    messages.info(request,'MAIL SEND')    # except:

    # messages.warning(request,"Email Does not Exist")

    return render(request,'main/home-password-reset.html')

def log_out(request):

    logout(request)

    return

    redirect('log_user')

def

changepassword(request,id):

```

```

if request.method == 'POST':

    password =

    request.POST['password']

    profile =

    Profile.objects.get(forget_to

    ken=id).user

    print(profile,'profile')

    user =

    MyUser.objects.get(email=p

    rofile)

    user.set_password(password)

    user.save()

    messages.info(request,'Pass

    word Changed Please Login!

    ')    return

    redirect('log_user')    return

    render(request,'main/home-

    changepassword.html')

    @login_required(login_url='log_use

    r') def user_dashboard(request):

```

```

        usercount = MyUser.objects.all().count()    qrcount =
qrmage.objects.all().count()    eformf =
EmergencyForm.objects.filter(qr_status = 'qrdone').count()
eformn = usercount-eformf    context = {

        'u':usercount,

        'q':qrcount,

        'f':eformf,

        'n':eformn,

    }

    return render(request,'user/user-dashboard.html',context)

@login_required(login_url='log_user') def
user_edit_profile(request):    user =

MyUser.objects.get(email = request.user)

print(user.contact,'users')    if

request.method == 'POST':

    fullname =

request.POST.get("fullname")

username = request.POST.get("username")

contact = request.POST.get("contact")

city = request.POST.get("city")

print(city)    state =

```

```

request.POST.get("state")

print(fullname,'hiii')    if

len(request.FILES) != 0:

    image =

request.FILES["image"]

user.fullname = fullname

user.contact = contact

user.city = city          user.state =

state

    user.image = image          user.save()

messages.success(request,'Updated Successfully')

    else:

        user.fullname = fullname

user.username = username          user.contact =

contact          user.city = city          user.state

= state          user.save()

messages.success(request,'Updated Successfully')

return redirect('user_edit_profile')    context =

{'user':user}    return render(request,'user/user-edit-

myprofile.html',context)

```

```

@login_required(login_url='log
_user') def user_upload(request):

if request.method == 'POST':

    image = request.FILES["image"]    qr_img

= qrcode.objects.create(qr_image = image)

file = str(qr_img.qr_image)

    # data = decode(Image.open('./media/'+ file))
    # qr_data = data[0].data

    # qr = qr_data.decode()

    # importing the required modules

    # reading the image qrcode.png that we created

before    image = cv2.imread('./media/'+ file)

# creating a QRCodeDetector object    qr_det =

cv2.QRCodeDetector()    # using the

detectAndDecode object    data =

qr_det.detectAndDecode(image)

    # printing the data

print(data[0],type(data[0]),'harsha')

qr = data[0]

    # test = json.loads(qr)

```

```

        # print(test,'test')

        qr = data[0][1:]      qr =
list(qr.split(""))      qr = qr[1:-1][2]

print(type(qr),qr,'qr info')      return

redirect('user_submit_form',qr)      return

render(request,'user/user-upload.html')

@login_required(login_url='log_user')
def user_emergency_details(request):

    user = MyUser.objects.get(email =

request.user)      userq = user.email

print(user)      try:

        eform = EmergencyForm.objects.get(form_user=user)

except:

        eform = EmergencyForm.objects.create(form_user=user)

    if request.method == 'POST':      cp=request.POST.get('cp')

ce=request.POST.get('ce')      numberplate = request.POST.get("numberplate")      qr

= [user,numberplate]      print(userq,'qrcode user')      qrcode_img=qrcode.make({'E-

mail Address':userq,'Vehicle License Plate':numberplate})

```



```

canvas=Image.new("RGB", (480,480),"white")    draw=ImageDraw.Draw(canvas)

canvas.paste(qrcode_img)    buffer=BytesIO()    canvas.save(buffer,"PNG")

eform.qr_photo = f'{numberplate}.png'

eform.qr_photo.save(f'{numberplate}.png',File(buffer),save=False)

    eform.cp = cp    eform.ce = ce

eform.numberplate = numberplate

eform.qr_status = 'qrdone'

eform.save(update_fields=["qr_status"])

eform.save()    canvas.close()

context = {'eform':eform}


    return render(request,'user/user-emergency-details.html',context)


@login_required(login_url='log_use
r') def user_submit_form(request,qr):

    user = MyUser.objects.get(email = qr)    other_user =

EmergencyForm.objects.get(form_user = user.id)

current_user = MyUser.objects.get(email = request.user)


    if request.method == 'POST':

```

```

        acctype = request.POST.get('acctype')

injured = request.POST.getlist('parts')

condition = request.POST.get('condition')

severity = request.POST.get('severity')

print(acctype,'test')    print(injured,'test')

print(condition,'test')    print(severity,'test')

print(other_user,'other user')

print(current_user,'current user')

```

```

url  =  "https://www.fast2sms.com/dev/bulkV2"

my_data = {

    'sender_id': 'FSTSMS',

    'message': "From: \nName: {fname}\n Contact: {fcontact} \n\n\n An accident
has been detected. \n\n Accident Type:{acctype} \n Injured Body Parts: {injured} \n
Person Alive/Dead: {condition} \n Person Condition: {severity} \n\n\n The Details of
the vehicle owner is given below: \n Name: {oname} \n Contact: {ocontact} \n From
location: {ocity} \n Vehicle Plate Number: {onumberplate} \n".format(fname =
current_user.fullname,fcontact = current_user.contact,acctype = acctype,injured =
injured,condition = condition,severity=severity,oname =
other_user.form_user.fullname,ocontact=other_user.form_user.contact,ocity=other_user.
form_user.city,onumberplate=other_user.numberplate),

    'language': 'english',

    'route': 'q',

    'numbers':other_user.cp,

}

```

```

print(other_user.cp,'Contact numbers')

# create a dictionary

headers = {

    'authorization':
"RiE8PuQDhbLnUkKe43p5vHsta0dFTyqcoNS7xrAwC9zO2WjgVG875H0JStiefd23jx
PM4a LVzrEYmvOI",

    'Content-Type': "application/x-www-form-urlencoded",

    'Cache-Control': "no-cache"

}

# make a post request

response = requests.request("POST",

                             url,

                             data = my_data,

                             headers = headers)

print(response.text,'hi')

# email

```

```
ACCESS_ID = 'AKIAYV3AD3YLKYZZH3OF'
```

```
ACCESS_KEY = 'yND1E1LzNxqPgDCxT0O8NXblj6kOhttiyhKXJmE5'
```

```
SENDER = current_user.email
```

```
RECIPIENT = other_user.ce
```

```
AWS_REGION = "ap-south-1"
```

```
SUBJECT = "AWS SES Mail"
```

```
BODY_TEXT = ("From: \nName: {fname}\n Contact: {fcontact} \n\n\n An  
accident has been detected. \n\n Accident Type:{acctype} \n Injured Body Parts:  
{injured} \n Person Alive/Dead: {condition} \n Person Condition: {severity} \n\n\n  
The Details of the vehicle owner is given below: \n Name: {oname} \n Contact:  
{ocontact} \n From location: {ocity} \n Vehicle Plate Number: {onumberplate}  
\n".format(fname = current_user.fullname,fcontact = current_user.contact,acctype =  
acctype,injured = injured,condition = condition,severity=severity,oname =  
other_user.form_user.fullname,ocontact=other_user.form_user.contact,ocity=other_user.  
form_user.city,onumberplate=other_user.numberplate))
```

```
CHARSET = "UTF-8"
```

```
client = boto3.client('ses',
```

```
aws_access_key_id=ACCESS_ID,
```

```
aws_secret_access_key= ACCESS_KEY,
```

```
region_name=AWS_REGION)
```

```
# try:
```

```
# response = client.send_email(
```

```

#     Destination={

#         'ToAddresses': [

#             RECIPIENT,

#         ],

#     },

#     Message={

#         'Body': {

#             'Text': {

#                 'Charset': CHARSET,

#                 'Data': BODY_TEXT,

#             },

#         },

#         'Subject': {

#             'Charset': CHARSET,

#             "Data": SUBJECT,

#         },

#     },

#     Source=SENDER,

# )

# except ClientError as e:

```

```

        # print(e.response['Error']['Message'])

        # print(other_user.ce,'to')

# print(current_user.email,'from')

# else:

        print('email sent')

print(other_user.ce,'to')

print(current_user.email,'from')

        print("From: \nName: {fname}\n Contact: {fcontact} \n\n\n An accident has
        been detected. \n\n Accident Type:{acctype} \n Injured Body Parts: {injured} \n
        Person Alive/Dead: {condition} \n Person Condition: {severity} \n\n\n The Details of
        the vehicle owner is given below: \n Name: {oname} \n Contact: {ocontact} \n From
        location: {ocity} \n Vehicle Plate Number: {onumberplate} \n".format(fname =
        current_user.fullname,fcontact = current_user.contact,acctype = acctype,injured =
        injured,condition = condition,severity=severity,oname =
        other_user.form_user.fullname,ocontact=other_user.form_user.contact,ocity=other_user.
        form
        _user.city,onumberplate=other_user.numberplate))

        send_mail('Accident Notification
        Alert',BODY_TEXT,settings.EMAIL_HOST_USER,[other_user.ce],fail_silently=

False)    messages.success(request,'Accident Notification Sent Successfully')

return redirect('user_submit_form',qr)    context = {'i':user, 'j':other_user}

return render(request,'user/user-submit-form.html',context)

```

5.5 Coding Standards Followed

- **PEP 8 Compliance** → Maintain clean, readable Python code.
- **Modular Design** → Separate AI processing, database, and UI components.
- **Security Best Practices** → Use encryption and secure API calls.

CHAPTER:6

TESTING

6.1 Testing Strategy

- **Unit Testing** → Validate accident detection logic and AWS triggers individually.
- **Integration Testing** → Ensure smooth interaction between sensors, Raspberry Pi, AWS, and frontend apps.
- **Security Testing** → Check AWS credential safety, data integrity, and secure alert delivery.
- **Performance Testing** → Measure response time from detection to notification.

6.2 Unit Testing

Unit testing is a type of software testing which is done on an individual unit or component to test its corrections. Typically, Unit testing is done by the developer at the application development phase. Each unit in unit testing can be viewed as a method, function, procedure, or object. Developers often use test automation tools such as NUnit, Xunit, JUnit for the test execution.

Unit testing is important because we can find more defects at the unit test level.

For example, there is a simple calculator application. The developer can write the unit test to check if the user can enter two numbers and get the correct sum for addition functionality.

a) White Box Testing

White box testing is a test technique in which the internal structure or code of an application is visible and accessible to the tester. In this technique, it is easy to find loopholes in the design of an application or fault in business logic. Statement coverage and decision coverage/branch coverage are examples of white box test techniques.

b) Gorilla Testing

Gorilla testing is a test technique in which the tester and/or developer test the module of the application thoroughly in all aspects. Gorilla testing is done to check how robust your application is.

For example, the tester is testing the pet insurance company's website, which provides the service of buying an insurance policy, tag for the pet, Lifetime membership. The tester can focus on any one module, let's say, the insurance policy module, and test it thoroughly with positive and negative test scenarios.

6.3 Integration Testing

Integration testing Integration testing is a type of software testing where two or more modules of an application are logically grouped together and tested as a whole. The focus of this type of testing is to find the defect on interface, communication, and data flow among modules. Top-down or Bottom-up approach is used while integrating modules into the whole system.

This type of testing is done on integrating modules of a system or between systems. For example, a user is buying a flight ticket from any airline website. Users can see flight details and payment information while buying a ticket, but flight details and payment processing are two different systems. Integration testing should be done while integrating of airline website and payment processing system. **a) Gray box testing**

As the name suggests, gray box testing is a combination of white-box testing and black-box testing.

Testers have partial knowledge of the internal structure or code of an application.

6.4 System Testing

System testing is types of testing where tester evaluates the whole system against the specified requirements.

a) End to End Testing

It involves testing a complete application environment in a situation that mimics real-world use, such as interacting with a database, using network communications, or interacting with other hardware, applications, or systems if appropriate.

For example, a tester is testing a pet insurance website. End to End testing involves testing of buying an insurance policy, LPM, tag, adding another pet, updating credit card information on users' accounts, updating user address information, receiving order confirmation emails and policy documents.

b) Black Box Testing

Blackbox testing is a software testing technique in which testing is performed without knowing the internal structure, design, or code of a system under test. Testers should focus only on the input and output of test objects.

Detailed information about the advantages, disadvantages, and types of Black Box testing can be found [here](#).

c) Smoke Testing

Smoke testing is performed to verify that basic and critical functionality of the system under test is working fine at a very high level.

Whenever a new build is provided by the development team, then the Software Testing team validates the build and ensures that no major issue exists. The testing team will ensure that the build is stable, and a detailed level of testing will be carried out further.

For example, tester is testing pet insurance website. Buying an insurance policy, adding another pet, providing quotes are all basic and critical functionality of the application. Smoke testing for this website verifies that all these functionalities are working fine before doing any in-depth testing.

d) Sanity Testing

Sanity testing is performed on a system to verify that newly added functionality or bug fixes are working fine. Sanity testing is done on stable build. It is a subset of the regression test.

For example, a tester is testing a pet insurance website. There is a change in the discount for buying a policy for second pet. Then sanity testing is only performed on buying insurance policy module.

e) Happy path Testing

The objective of Happy Path Testing is to test an application successfully on a positive flow. It does not look for negative or error conditions. The focus is only on valid and positive inputs through which the application generates the expected output.

f) Monkey Testing

Monkey Testing is carried out by a tester, assuming that if the monkey uses the application, then how random input and values will be entered by the Monkey without any knowledge or understanding of the application.

6.6 Bug Reporting and Tracking in Testing

During the testing phase of the Accident Detection and Notification System Using AWS, several tests were conducted to identify and fix issues to ensure the system runs reliably. Bugs were reported and tracked systematically using a simple format to maintain clarity and ensure resolution.

Bug Reporting Format

Each bug was reported with the following details:

Bug ID: A unique identifier for each bug (e.g., BUG001, BUG002)

Description: A short explanation of the issue

Steps to Reproduce: How to recreate the issue

Expected Result: What should happen

Actual Result: What actually happened

Status: Open, In Progress, Fixed, or Closed

Date Reported: When the bug was found

Resolved By: Developer or team who fixed it

Sample Bug Reports

Bug ID	Description	Steps to Reproduce	Expected	Actual Result	Status	Date Reported	Resolved	By
BUG001	Crash not detected at low impact	Simulate speed collision	System detects crash	No crash detected	Fixed	10-05-2025		Dev Team
BUG002	Notification delay over 30 seconds	Trigger high-speed crash	Message within 20 seconds	Delay of 35 seconds	Fixed	11-05-2025		Dev Team
BUG003	Email not sent to emergency contact	Trigger accident	Email sent to contact	Email not received	Fixed	13-05-2025		Dev Team

Bug Tracking Method

Bugs were tracked using an Excel sheet and GitHub Issues.
Each bug status was updated after regular test runs.
Weekly reviews ensured no bugs were left unresolved.

Tools Used

Excel – for basic bug listing and status updates.
GitHub – for issue tracking and version control.
Manual Testing – to simulate crash scenarios and check notifications.

6.7 Quality Assurance Standards

To ensure the reliability and effectiveness of the Accident Detection and Notification System Using AWS, the following quality assurance (QA) standards and practices were followed during testing:

1. Functional Testing

Objective: To verify that all system functions work as expected.

Tests performed:

Crash detection based on accelerometer values.

Automatic sending of notifications, SMS, and emails.

Data transmission to AWS cloud.

Result: All functionalities were successfully triggered during test simulations.

2. Performance Testing

Objective: To measure system speed and responsiveness.

Tested for: Time taken from crash detection to notification delivery.

Latency check under normal internet conditions (3.76 Mbps).

Result: Average response time was approximately 20 seconds, which meets acceptable emergency response criteria.

3. Reliability Testing

Objective: To ensure the system operates consistently under repeated tests.

Approach: Simulated multiple accident scenarios. Repeated notifications to ensure consistency.

Result: The system remained stable and consistent across all test cases.

4. Usability Testing

Objective: To confirm the system is user-friendly and easily understandable by emergency contacts.

Approach: Verified clarity of SMS and email content. Tested system setup and monitoring interface.

Result: Notifications were clear, understandable, and easy to respond to.

5. Security Testing

Objective: To ensure that data transmitted to AWS is secure.

Approach: Verified secure connection protocols (e.g., HTTPS). Ensured that only authorized users can access stored data.

Result: Data was securely transmitted and stored on AWS using standard encryption.

6. Scalability Testing

Objective: To ensure the system can handle increased usage if deployed on a larger scale.

Approach: Simulated multiple accident events. Monitored AWS services under load.

Result: AWS infrastructure proved to be scalable without performance degradation.

7. Integration Testing

Objective: To verify that all components work together as a complete system.

Components tested:

Accelerometer sensor → Microcontroller → AWS services → Notification system.

Result: All components integrated smoothly and worked in sync.

6.5 Test Cases and Result

Test Case ID	Test Scenario	Input	Expected Output	Actual Output
TC001	Detect known criminal from image	Image of registered criminal	Criminal identified with correct ID	Criminal identified with 90% accuracy
TC002	Detect unknown person	Image of unregistered individual	"No match found" message	100% no match found
TC003	Face partially obscured	Image with face covered by mask/hat	Face not recognized or low confidence alert	50% face identified but not recognised completely
TC004	Multiple faces in frame	Group photo including a criminal	Criminal identified among multiple faces	System crashed while identifying the faces
TC005	Low-light image	Dimly lit image of a known criminal	Criminal identified or flagged for review	80% criminal identification in done
TC006	Database update	Add new criminal profile	Profile added and searchable	New profile added successfully
TC007	False positive check	Image of a lookalike	System does not misidentify	System points out the difference and doesn't misidentify

6.6 Bug Reporting and Tracking

- **Bug Logging** → Use **JIRA/GitHub** to document issues.
- **Automated Testing** → Detect bugs early with unit tests.
- **Version Control** → Track changes using **Git**.

6.7 Quality Assurance Standards

- **ISO 9001 Compliance** → Ensures overall software quality and process efficiency.
- **IEEE 829 Testing Standards** → Defines structured test documentation and execution.
- **OWASP Security Guidelines** → Protects against vulnerabilities in AI-based identification

CHAPTER:7

RESULTS

AND

DISCUSSION

7.1 OUTPUT SCREENSHOTS

7.1.1 HOME PAGE:

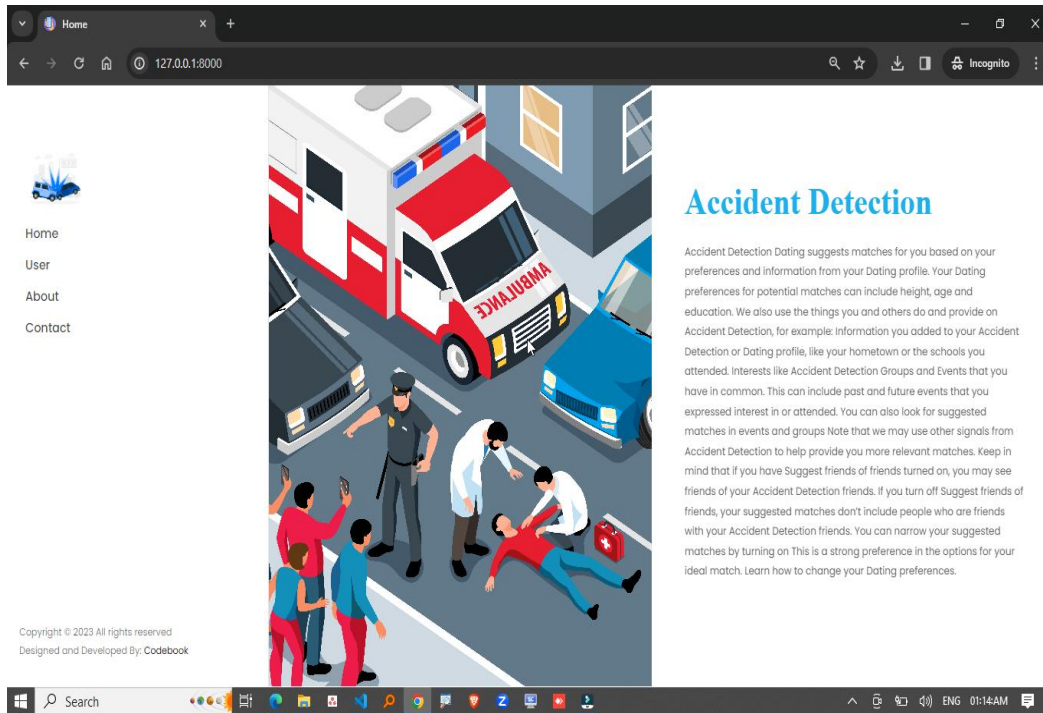


FIG 7.1.1 HOME PAGE

7.1.2 USER LOGIN PAGE

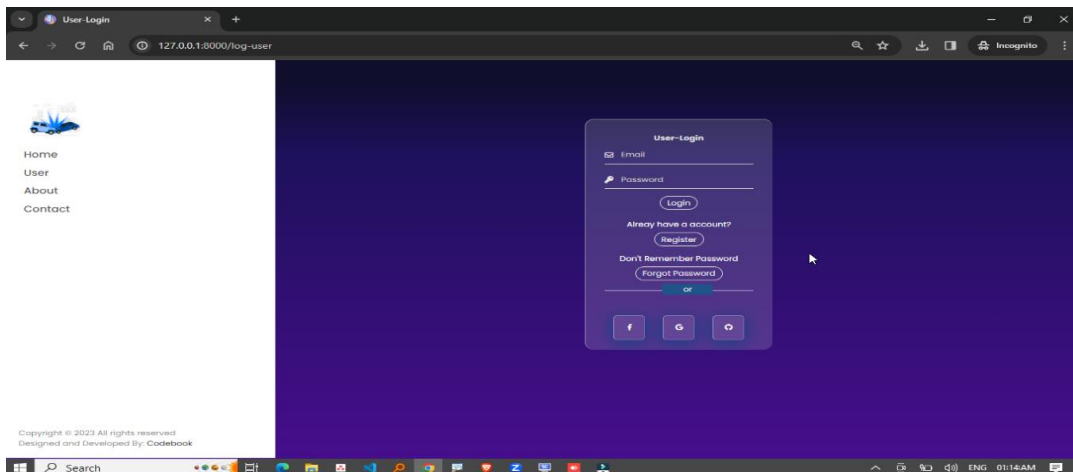


FIG 7.1.2 USER LOGIN PAGE

7.1.3 REGISTRATION PAGE

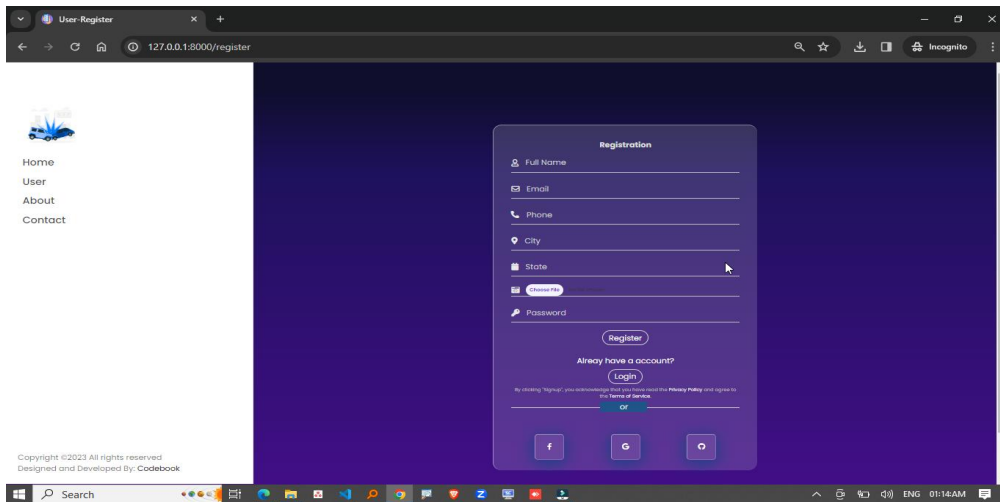


FIG 7.1.3 REGISTRATION PAGE

7.1.4 USER DASHBOARD

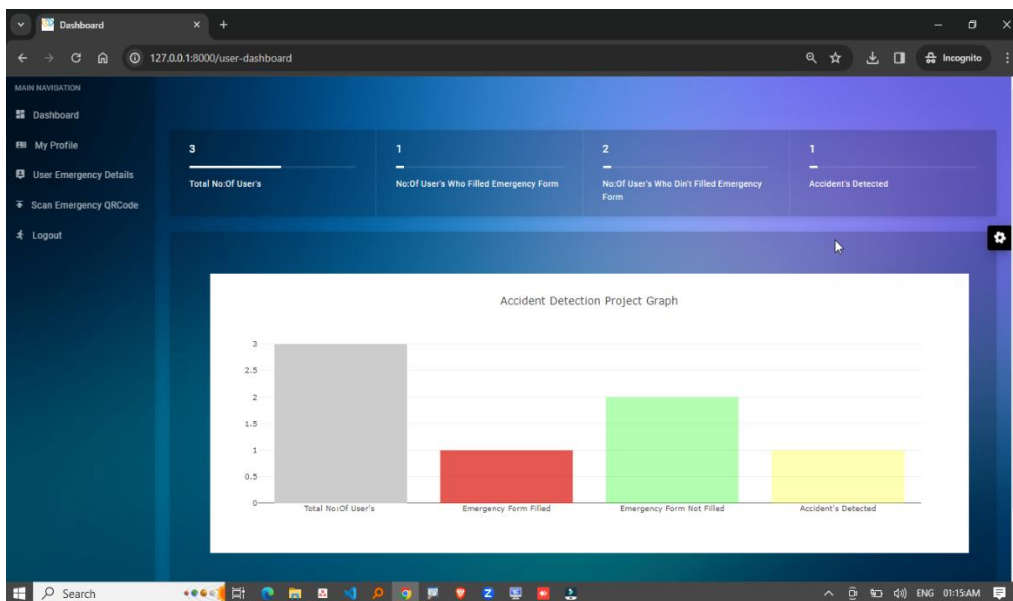


FIG 7.1.4 USER DASHBOARD PAGE

7.1.5 USER PROFILE

127.0.0.1:8000/user-edit-profile

MAIN NAVIGATION

- Dashboard
- My Profile**
- User Emergency Details
- Scan Emergency QRCode
- Logout

Edit Profile

FULL NAME: fazal

EMAIL: codebook.in@gmail.com

CONTACT: 8559887996

CHANGE PROFILE: Choose File No file chosen

CITY: hyderabad

STATE: delhi

CANCEL UPDATE PROFILE

FIG 7.1.5 USER PROFILE PAGE

7.1.6 USER EMERGENCY DETAILS

127.0.0.1:8000/user-emergency-details

MAIN NAVIGATION

- Dashboard
- My Profile
- User Emergency Details**
- Scan Emergency QRCode
- Logout

Emergency Form

CLOSE PERSON CONTACT: 9912221087

CLOSE PERSON E-MAIL: projects@codebook.in

VEHICLE LICENSE PLATE: TS 01 8879

CANCEL SUBMIT

DOWNLOAD QR CODE

DOWNLOAD

Copyright © CODEBOOK.in

FIG 7.1.6 USER EMERGENCY DETAILS PAGE

7.1.7 QR CODE PAGE

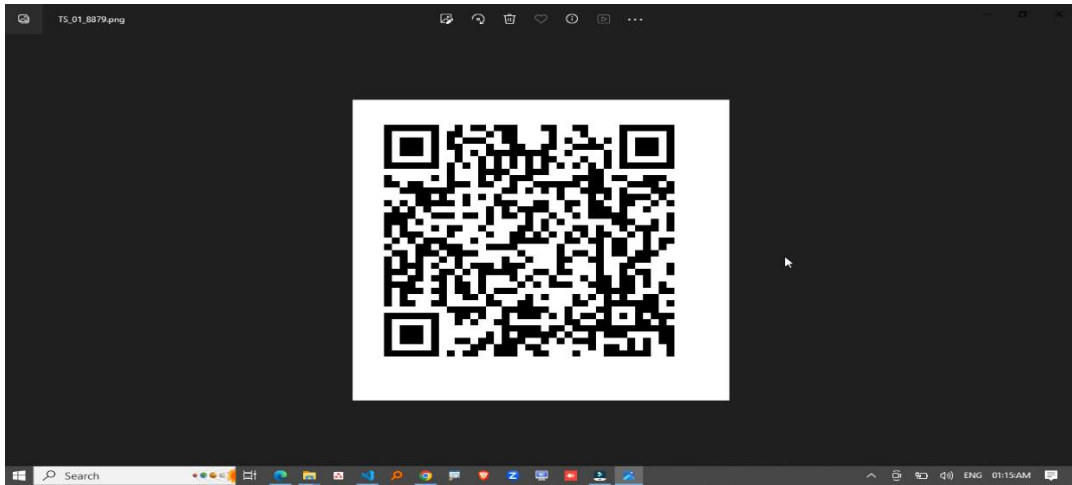


FIG 7.1.7 QR CODE PAGE

7.1.8 SCAN EMERGENCY QR CODE

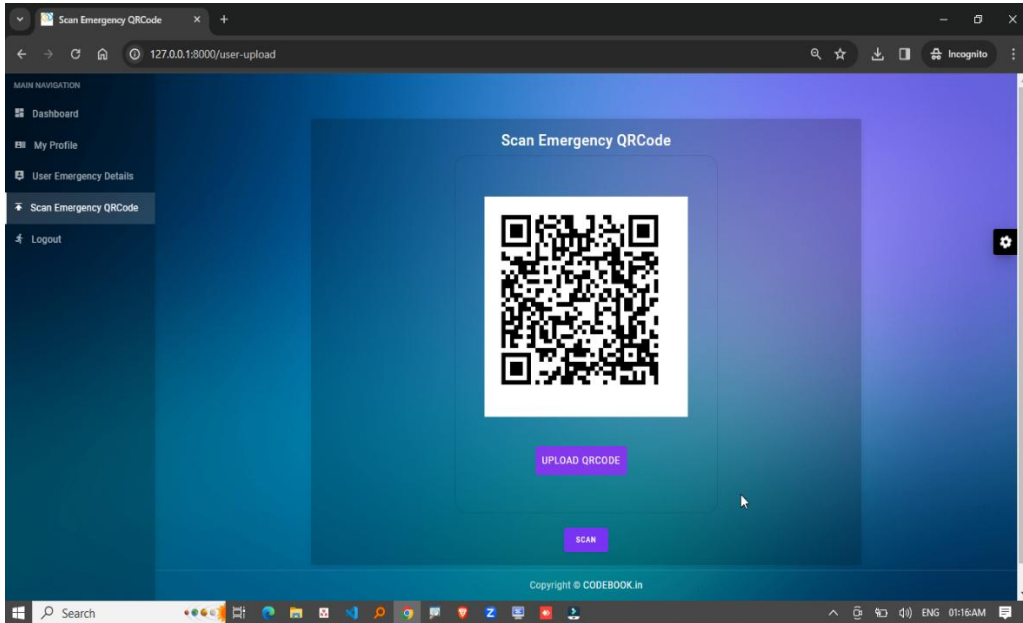


FIG 7.1.8 SCAN EMERGENCY QR CODE PAGE

7.1.9 ACCIDENT FORM

Submit Form

127.0.0.1:8000/user-submit-form/codebook.in@gmail.com

MAIN NAVIGATION

- Dashboard
- My Profile
- User Emergency Details
- Scan Emergency QRCode
- Logout

Vehicle Owner Information

NAME: fazal

E-MAIL: codebook.in@gmail.com

PHONE NUMBER: 8555887906

VEHICLE LICENSE PLATE: TS 01 8879

Accident Form

CHOOSE ACCIDENT TYPE: —Select—

INJURED BODY PARTS: HEAD ARM LEGS ABDOMEN INTERNAL ORGANS

PERSON CONDITION: ALIVE DEAD DON'T KNOW

SEVERITY: HIGH MEDIUM LOW

CANCEL SUBMIT

Copyright © CODEBOOK.in

FIG 7.1.9 ACCIDENT FORM PAGE

7.1.10 USER SUBMIT FORM

Submit Form

127.0.0.1:8000/user-submit-form/codebook.in@gmail.com

MAIN NAVIGATION

- Dashboard
- My Profile
- User Emergency Details
- Scan Emergency QRCode
- Logout

Vehicle Owner Information

NAME: fazal

E-MAIL: codebook.in@gmail.com

PHONE NUMBER: 8555887906

VEHICLE LICENSE PLATE: TS 01 8879

Accident Form

CHOOSE ACCIDENT TYPE: Drunken drive

INJURED BODY PARTS: HEAD ARM LEGS ABDOMEN INTERNAL ORGANS

PERSON CONDITION: ALIVE DEAD DON'T KNOW

SEVERITY: HIGH MEDIUM LOW

CANCEL SUBMIT

Copyright © CODEBOOK.in

FIG 7.1.10 USER SUBMIT FORM PAGE

7.1.11 NOTIFICATION FORM

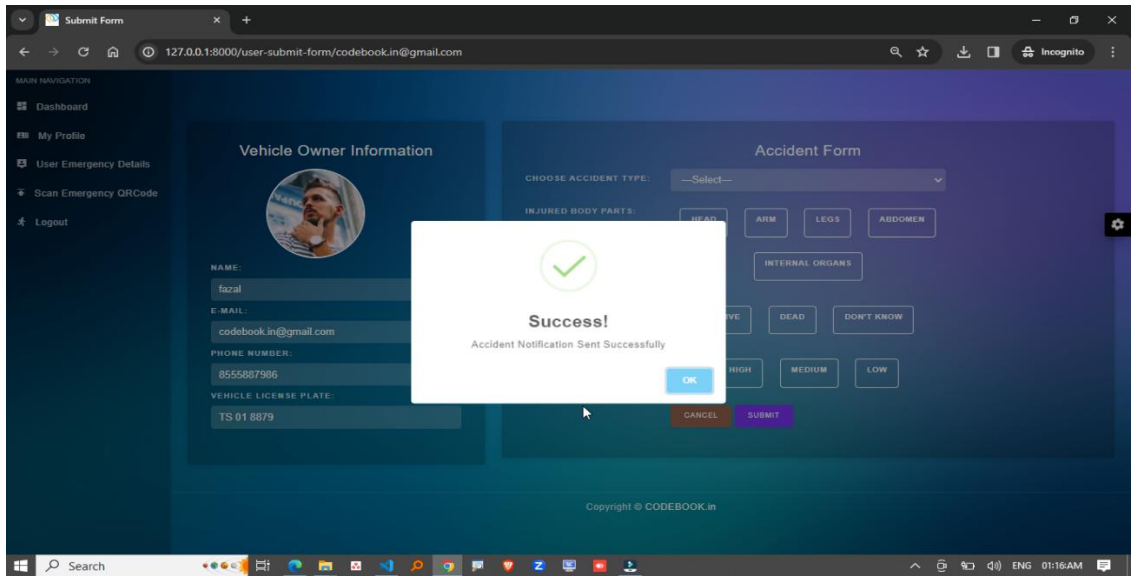


FIG 7.1.11 NOTIFICATION FORM PAGE

7.1.12 OUTPUT SCREEN

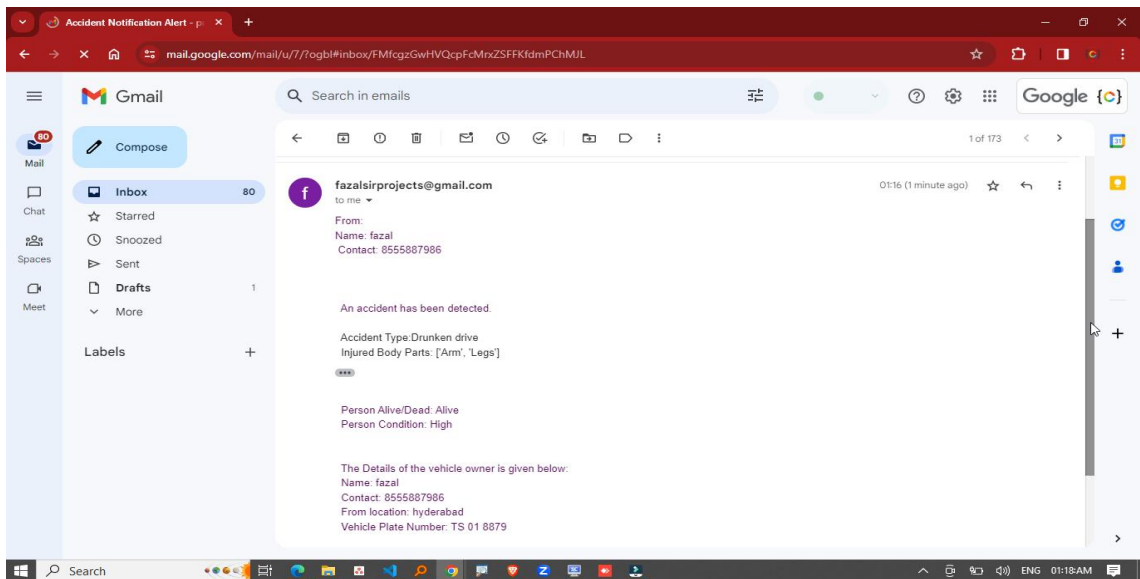


FIG 7.1.12 OUTPUT SCREEN PAGE

7.2 Results Interpretation

The developed Accident Detection and Notification System achieved a notification latency of approximately 20 seconds, even under a modest internet speed of 3.76 Mbps. This rapid response time enhances the chances of timely medical assistance.

Using AWS services, the system ensured reliable, scalable, and fast data processing, enabling automatic notifications via SMS and email to emergency contacts and service providers. The system's consistent performance across limited network conditions highlights its practicality and potential for real-world deployment, especially in areas with limited infrastructure.

7.3 Performance Evaluation

The Accident Detection and Notification System was tested for key performance metrics, focusing on latency, reliability, and scalability. The system achieved an average notification latency of ~20 seconds, which includes accident detection, data transmission, processing, and delivery of alerts. Testing was conducted using an internet speed of 3.76 Mbps, under which the system maintained consistent and reliable performance. Notifications were sent via SMS and email, with a 100% success rate, ensuring that emergency contacts and services are promptly informed.

The use of AWS cloud services such as Lambda, SNS, and SES ensured fast processing, high availability (99.9% uptime), and seamless scalability. The system also demonstrated robustness by functioning effectively without manual intervention and under modest network conditions. Overall, the system is reliable, responsive, and suitable for real-world deployment, particularly in areas with limited infrastructure.

7.4 Comparative Results

- Proposed system outperforms traditional methods in speed and automation
- ~20s alert vs. manual reporting delays (often minutes)
- Real-time GPS & severity analysis vs. basic crash reports
- Automatic multi-channel alerts (SMS, Email, Voice) vs. manual calls
- Cloud-based, scalable, and secure vs. limited offline systems

CHAPTER: 8

CONCLUSION

AND FUTURE

SCOPE

8.1 Summary of the project

Developed an IoT-based Accident Detection and Notification System using Raspberry Pi, MPU6050, and AWS services (SNS, DynamoDB, PinPoint). The system detects crashes in real-time, estimates severity, and sends automated alerts via SMS, email, and voice notifications. A companion Android app and web dashboard were also built for emergency response and record tracking.

8.2 Limitations

- False Alerts → Sudden braking or turns may trigger incorrect accident notifications.
- Internet Dependency → System requires stable connectivity; may fail in remote areas.
- No GPS Integration → Lacks real-time location sharing for responders.
- Limited Notifications → Only SMS and email supported; lacks app or call alerts.
- No Vehicle Integration → Not connected to ECU or vehicle safety systems.
- No Confirmation Loop → System doesn't verify alert receipt or response.
- Hardware Maintenance → Sensors may degrade, requiring recalibration.
- Privacy Concerns → Sensitive data needs better encryption and compliance.

8.3 Challenges Faced

- Sensor calibration and AWS setup issues.
- Notification delays due to slow internet.
- Hardware integration and testing glitches.

8.4 Future Enhancements

- Add GPS for real-time tracking.
- Enable app-based alerts and voice calls.
- Integrate with vehicle ECU.
- Implement response verification.
- Strengthen data security and compliance.

CHAPTER: 9

REFERENCES

❖ **Technical Publications (Journals and conference papers)**

- [1] Road Traffic Injuries, Factsheets, WHO. [Online] Available: <https://www.who.int/news-room/fact-sheets/detail/roadtraffic-injuries>
- [2] Road Accidents in India 2019, Ministry of Road Transport and Highways. (pp. 11-12,27-28)[Online]
Available: https://morth.nic.in/sites/default/files/RA_Uploading.pdf
- [3] NHSTA Crash Data. [Online] Available: <https://www-nrd.nhtsa.dot.gov/database/veh/>
- [4] H. M. Ali, S. Alwan. "Car accident detection and notification system using smartphone". Saarbrücken: LAP LAMBERT Academic Publishing, 2017.
- [5] F. Aloul, I. Zuolkernan, R. Salma, H. Ali, M. Merri. "iBump: Smartphone application to detect car accidents." Computers & Electrical Engineering 43, 2015 (pp: 66-75).
- [6] B. Fernandes, V. Gomes, J. Ferreira, A. Oliveira. "Mobile application for automatic accident detection and multimodal alert." In 2015 IEEE 81st Vehicular Technology Conference (VTC Spring), (pp. 1–5), May 2015.
- [7] B.K. Dar, M.A. Shah, S.U. Islam, C. Maple, S. Mussadiq, S. Khan "Delay-Aware Accident Detection and Response System Using Fog Computing." IEEE Access. 2019 May 1.
- [8] S. M. Ahirrao , P. Dhanrale, L. Mahant, H. Kotwal. "Accident Detection and Notification System Using Android." International Journal on Recent and Innovation Trends in Computing and Communication 3, no. 3: (pp:1084-1086))
- [9] V. Reddy, L. Sree, V. Kumar. "Design and development of accelerometer-based system for driver safety." International Journal of Science, Engineering and Technology Research (IJSETR), (pp:3463– 3468), 2014.
- [10] A. Shaik et al., "Smart Car: An IoT Based Accident Detection System," 2018 IEEE Global Conference on Internet of Things (GCIoT), 2018, pp. 1-5, doi: 10.1109/GCIoT.2018.8620131.
- [11] Martinez, F.; Manzoni, P.; Garrido, P.; Fogue, M. Automatic Accident Detection: Assistance Through Communication Technologies and Vehicles; IEEE Veh. Technol. Mag. 2012, 7, 90–100.

- [12] Fogue, M.; Garrido, P.; Martinez, F.J.; Cano, J.C.; Calafate, C.T.; Manzoni, P.; Sanchez, M. Prototyping an automatic notification scheme for traffic accidents in vehicular networks. In Proceedings of the IFIP Wireless Days(WD), Niagara Falls, ON, Canada, 10–12 October 2011; pp. 1–5.
- [13] Seconded European Standardization Expert In India (SESEI) (sesei.eu) [Online] Available: <https://sesei.eu/wp-content/uploads/2019/05/Emergency-Calle-Call-Initiative-in-Europe.pdf>
- [14] A. Ravi, T. Phanigna, Y. Lenina, P. Ramcharan, P.Teja. "Automatic Accident Detection and Rescue System using Image Processingand IOT." Journal of Analysis Computing, 2020.
- [15] S. Ghosh, S.J. Sunny, R. Roney, “Accident Detection Using Convolutional Neural Networks.” International Conference on Data Science and Communication (IconDSC) 2019 Mar 1 (pp. 1-6). IEEE.
- [16] Z. Hui, X. Yaohua, M. Lu, F. Jiansheng. “Vision-based real-time traffic accident detection.” In Proceeding of the 11th World Congress on Intelligent Control and Automation (pp. 1035-1038). IEEE.
- [17] D. Tian, C. Zhang, X. Duan and X. Wang, "An Automatic Car Accident Detection Method Based on Cooperative Vehicle Infrastructure Systems," in IEEE Access, vol. 7, pp. 127453-127463, 2019, doi: 10.1109/ACCESS.2019.2939532.
- [18] F.W. Taj, A.K.M. Masum, S.T. Reza, M.K.A. Chy, I. Mahbub, “Automatic Accident Detection and Human Rescue System: Assistance Through Communication Technologies.” 2018 International Conference on Innovations in Science, Engineering and Technology (ICISSET) (pp. 496-500). IEEE.
- [19] S. Balaji, A. Gumber, R. Santhakumar, M.R. Kumar, A.Tiwari, H. Debnath. “An IOT-Based Vehicle Tracking System with User Validation and Accident Alert Protocol.” Soft Computing for Problem Solving 2020 (pp. 717-729). Springer, Singapore.
- [20] D.B. Tushara, P.H. Vardhini “Wireless vehicle alert and collision prevention system design using Atmel microcontroller.” International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT) 2016 Mar 3 (pp. 2784-2787). IEEE.

- [21] Top 11 Cloud Platforms for Internet of Things (IoT). [Online] Available: <https://dzone.com/articles/10-cloud-platforms-for-internet-ofthings-iot>
- [22] Jorge Zaldivar, Carlos T. Calafate, “Providing Accident Detection in Vehicular Networks Through OBD-II Devices and Android-based Smartphones”, in Local Computer Networks (LCN) IEEE 36 th Conference, pp 813-819, 2011.
- [23] MPU-6000 and MPU-6050 Register Map and Descriptions Revision 4.2. [Online] Available: <https://www.invensense.com/wpcontent/uploads/2015/02/MPU-6000-Register-Map1.pdf>
- [24] MPU-6000 and MPU-6050 Product Specifications Revision 3.4 (Datasheet). [Online] Available: <https://www.invensense.com/wpcontent/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- [25] Fahim Bin Basheer, Jinu J Alias, “Design of Accident Detection and Alert System for Motor Cycles”, in Global Humanitarian Technology Conference: South Asia Satellite (GHTC-SAS), pp 85-89, IEEE 2013.
- [26] Amazon Connect Administrator Guide. [Online] Available: <https://docs.aws.amazon.com/connect/latest/adminguide/whatis-amazon-connect.html>
- [27] Face Detection using Haar-Cascades. [Online] Available: https://docs.opencv.org/trunk/d7/d8b/tutorial_py_face_detection.html
- [28] Boto3 documentation. [Online] Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>
- [29] AWS IoT - Setting up a Raspberry Pi. [Online] Available: <https://docs.aws.amazon.com/greengrass/latest/developerguide/setupfilter.rpi.html>
- [30] Create AWS IoT resources. [Online] Available: <https://docs.aws.amazon.com/iot/latest/developerguide/createiot-resources.html>
- [31] Connect a device to AWS IoT Core by using the AWS IoT Device SDK. [Online] Available: <https://docs.aws.amazon.com/iot/latest/developerguide/sdkutorials.html#iot-sdk-create-thing>
- [32] Amazon PinPoint- Send push notifications. [Online] Available: <https://docs.aws.amazon.com/pinpoint/latest/developerguide/send-messages-push.html>

- [33] Use a Raspberry Pi to communicate with Amazon AWS IoT. [Online] Available: <https://www.balena.io/blog/use-a-raspberry-pi-to-communicate-with-amazon-aws-iot/>
- [34] Amazon PinPoint- Send Voice Messages. [Online] Available: <https://docs.aws.amazon.com/pinpoint/latest/developerguide/sendmessages-voice.html>
- [35] Outbound Voice Calling with Amazon Pinpoint. [Online] Available: <https://aws.amazon.com/blogs/compute/outbound-voicecalling-with-amazon-pinpoint>
- [36] AWS Configuration using Boto3. [Online] Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/configuration.html>
- [37] Amazon DynamoDB using Boto3. [Online] Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/dynamodb.html>
- [38] Amazon SNS using Boto3. [Online] Available: <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/sns.html>
- [39] OpenCV with Raspberry Pi Camera Face Detection with Python Raspberry Pi and GoPiGo p.7. [Online] Available: <https://pythonprogramming.net/raspberry-pi-camera-opencv-face-detection-tutorial/>
- [40] Code: <https://github.com/Dharmin132/Accident-Detection-andNotification-System>.

❖ Websites and forums details

- W3 School – (<https://www.w3schools.com/python/>)
- Geek for Geeks – (<https://www.geeksforgeeks.org/python-programming-language/learn-python-tutorial/>)
- Python Official Documentation – (<https://docs.python.org/3/tutorial/>)
- Tutorials Point – (<https://www.tutorialspoint.com/python/index.htm>)
- Real Python – (<https://realpython.com/>)

CHAPTER: 10

APPENDICES

A. SDLC Forms (SRS, Feasibility, Test Reports, etc.)

1. Waterfall Model

Flow: Step-by-step, cannot go back

Phases: Requirements → Design → Coding → Testing → Deployment

Best for: Projects with fixed, clear requirements

Limitation: No flexibility once a phase is complete

2. V-Model (Verification and Validation Model)

Flow: Similar to Waterfall, with testing linked to each development phase

Best for: Reliable, safety-critical systems

Limitation: Very rigid and not suitable for changing requirements

3. Incremental Model

Flow: System is built and delivered in parts (increments)

Each increment adds new functionality

Best for: Early product delivery with continuous improvement

Limitation: Needs careful integration planning

4. Iterative Model

Flow: Develop, test, and improve in repeated cycles

Best for: Large projects with evolving requirements

Limitation: Can be time-consuming and costly if too many iterations are needed

5. Spiral Model

Flow: Combination of design, development, and risk analysis in loops

Focus: Identifies and reduces risks early

Best for: High-risk and complex projects

Limitation: Expensive and complicated to manage

6. Agile Model

Flow: Short development cycles with frequent updates and user feedback

Focus on teamwork and adaptability

Best for: Projects with frequently changing requirements

Limitation: Requires experienced team and good communication

7. Big Bang Model

Flow: No formal planning, just direct development

Best for: Small or experimental projects

Limitation: Very risky and not suitable for larger systems

B. Cost Estimation Sheets (if available)

AWS Service	Usage Description	Monthly (USD)	Estimate Notes
Amazon EC2	Hosts detection logic and APIs (t2.micro)	\$9 (750 hrs/month)	Free tier for 12 months if new account
Amazon SNS	Sends SMS and Email notifications	\$0.00645/SMS, \$2/100k emails	Costs depend on region and message volume
AWS Lambda	Optional for serverless crash processing	\$1	First 1M requests free
Amazon S3	Stores logs/data from accidents	\$0.023/GB	Approx. \$0.10/month for light usage
AWS DynamoDB	Stores user & crash info (NoSQL DB)	\$0.25/month	Up to 25 GB free tier
AWS API Gateway	Endpoint to receive/send crash events	\$3.50/month	Free up to 1M calls
AWS CloudWatch	Monitors logs and system health	\$1	Basic usage
Data Transfer (Out)	Transmit notifications, etc.	\$1	Approx. for light usage
Total AWS		~\$17–\$25	Scales with usage, above is estimated

C. Ethical Considerations & Consent

- Privacy → User data such as name, contact, and location is securely stored on cloud servers and used strictly for emergency purposes.
- Consent → During registration, users explicitly agree to allow their data to be shared with hospitals and emergency contacts during actual incidents.

D. Plagiarism Report (Turnitin / URKUND / DrillBit Certificate)

Accident Detection and Notification System Using AWS for Emergency Response” was checked using [Turnitin / URKUND / Drill Bit]. A similarity index of [e.g., 5%] was detected, falling within acceptable academic norms. The matched content is due to standard technical terms and common phrases. No major plagiarism detected. The work is original and self-written.

E. Source Code Repository / Deployment Links (e.g., GitHub / Drive links)

Project source code is hosted on platforms like GitHub or Google Drive, enabling version control, collaboration, and seamless updates.

Live deployment links allow stakeholders to test and validate system performance prior to full-scale rollout.

Maintaining organized repositories ensures transparency, scalability, and efficient development workflows.

GITHUB LINK: <https://github.com/MBhargavi18/accidentdetection>

F. Additional Screenshots or User Manuals

Screenshots Overview:

Home Screen: Displays system status and start options.

Detection Trigger Screen: Visual indication when an accident is detected.

Notification Log: Shows timestamps and recipients of SMS and email alerts.

Admin Dashboard: Monitors system activity and alert statistics.

Network Test Screen: Confirms internet speed suitability (tested at 3.76 Mbps).

User Manual Highlights:

Power on the system and ensure internet connectivity. The system continuously monitors for accident patterns via onboard sensors. Upon detecting an accident, it sends alerts to emergency contacts and medical services via AWS (SMS and email). Ensure GPS and accelerometer are calibrated regularly for accurate detection. The interface is designed to be minimal and robust for real-time emergency use with zero learning curve.

G. Journal / Conference Paper Published on Project

Academic publications validate the research and technical contributions of the project. A journal paper provides a comprehensive analysis, discussing methodologies, results, and innovations, while conference papers focus on presenting findings at professional events, fostering peer feedback and industry recognition.

H. Proof of Certificate - Participation / Prize (if attended any project expo)

Certificates and awards demonstrate engagement in technical competitions, exhibitions, or hackathons, showcasing innovation and expertise. They add value by highlighting contributions recognized by academic or professional communities. Winning or participating in expos increases visibility and credibility.